TECHNICAL REPORT

Retail Demand Intelligence Platform

An Agentic AI System for Multi-Store Demand Forecasting, Uncertainty-Aware Anomaly Detection, and Explainable Decision Support

By

Karthik

July 2026

This report documents the design, implementation, and evaluation of a full-stack demand forecasting, anomaly detection, and agentic decision-support system built end-to-end over a schema-realistic multi-store retail dataset.

Abstract

This report presents the design, implementation, and evaluation of a retail demand intelligence platform that unifies four capabilities usually built and shipped separately: multi-model demand forecasting with per-series model selection, decomposition-based anomaly detection with explicit explained/unexplained separation, calibration-checked prediction intervals, and an agentic, tool-calling natural-language interface with a persisted audit trail. The system forecasts 8 weeks ahead for 240 store/department series drawn from a 62,400-row, schema-realistic synthetic retail dataset, selecting between a seasonal-naive baseline, a classical Holt-Winters model, and a global gradient-boosted model on a per-series basis using rolling-origin backtesting. The gradient-boosted model achieves a mean weighted absolute percentage error (WAPE) of 8.21% and wins the backtest on 140 of 240 series, narrowly ahead of the seasonal-naive baseline (8.58%, 90 series), a result reported honestly rather than rounded in the more sophisticated model's favor. An anomaly detector built on robust, trailing-window statistical decomposition correctly separates promotional and calendar-holiday-driven spikes from genuinely unexplained demand shocks, and is validated directly against injected ground-truth events: 100% of synthetic data-quality errors, 211 of 240 series affected by an injected regime shock, and 4 of 5 weeks of a localized, unexplained supply disruption. Every forecast additionally carries an 80% prediction interval, whose real, held-out empirical coverage (74.7%) is measured and reported rather than assumed. Local and global model explainability are implemented via occlusion-based attribution and permutation importance as disclosed substitutes for SHAP, which could not be installed in the target build environment. Finally, a tool-calling agent – runnable against a deterministic mock planner or real Anthropic and OpenAI backends behind a shared tool registry – answers natural-language operational questions, simulates hypothetical markdown and holiday scenarios by live model recomputation, and logs a complete, auditable reasoning trace for every answer it gives. The system is validated by a 49-test automated suite, and every design decision, substitution, and limitation encountered during development is disclosed in place rather than omitted.

Keywords: demand forecasting, time series, gradient boosting, rolling-origin backtesting, anomaly detection, prediction intervals, model explainability, agentic artificial intelligence, tool-calling large language models, retail analytics.

Contents

Abstract i

List of Figures v

List of Tables vi

1 Introduction 1

1.1 Background and Motivation . . . 1
1.2 Problem Statement . . . 1
1.3 Research Objectives . . . 2
1.4 Scope and Limitations . . . 2
1.5 Report Structure . . . 3

2 Related Work 4

2.1 Retail Demand Forecasting . . . 4
2.2 Gradient-Boosted Trees for Tabular Forecasting . . . 4
2.3 Anomaly Detection in Retail Time Series . . . 4
2.4 Prediction Intervals and Uncertainty Quantification . . . 5
2.5 Model Explainability . . . 5
2.6 Agentic AI and Tool-Calling Systems . . . 5
2.7 Gaps Addressed by This System . . . 6

3 System Design and Architecture 7

3.1 Architectural Overview . . . 7
3.2 Data Pipeline and Feature Engineering . . . 8
3.3 Forecasting Models . . . 8
3.4 Anomaly Detection Design . . . 9
3.5 Prediction Interval Design . . . 10
3.6 Explainability Design . . . 10
3.7 Agent Architecture and Tool Design . . . 10
3.8 Dashboard Frontend . . . 11

4 Implementation **12**
4.1 Development Environment . . . 12
4.2 Synthetic Data Generation . . . 12
4.3 Feature Engineering Implementation . . . 13
4.4 Rolling-Origin Backtest Implementation . . . 13
4.5 Prediction Interval Implementation . . . 13
4.6 Explainability Implementation . . . 14
4.7 What-If Scenario Simulation . . . 14
4.8 Agent Assembly and Tool Wiring . . . 14
4.9 API and Frontend Implementation . . . 15
4.10 Testing and Validation . . . 15

5 Evaluation and Results **16**
5.1 Evaluation Framework . . . 16
5.2 Forecasting Accuracy . . . 16
5.3 Anomaly Detection Validation . . . 18
5.3.1 A Limitation Found and Fixed . . . 20
5.4 Prediction Interval Calibration . . . 20
5.5 Feature Importance and Explainability Results . . . 21
5.6 Production Monitoring Simulation . . . 21
5.7 Dashboard and Agent Demonstration . . . 22

6 Discussion **24**
6.1 Interpretation of Findings . . . 24
6.2 Comparison with Related Approaches . . . 24
6.3 Practical Implications . . . 24
6.4 Limitations . . . 25
6.5 Data and Ethical Considerations . . . 25

7 Conclusion and Future Work **26**
7.1 Conclusion . . . 26
7.2 Future Work . . . 26

Bibliography **27**

Appendix **28**

A Repository Structure **29**

B Agent Tool Registry **30**

List of Figures

3.1 End-to-end system architecture. The dashed line separates the offline batch stage (data generation, backtesting, anomaly scanning, forward forecasting, and monitoring simulation) from the thin online serving layer (the agent and the Flask API/-dashboard), which only ever reads artifacts the batch stage already wrote. 8

5.1 Mean backtested WAPE by model across all 240 series. 17
5.2 Number of series (of 240) on which each model achieved the lowest backtested WAPE. 17
5.3 Example series (Store 3, Department 5): trailing 60 weeks of actuals, two flagged anomalies, and the 8-week forward forecast under the selected global gradient-boosted model. 18
5.4 The same injected macro shock produces opposite department-level responses – one department spikes while another crashes – which is why the anomaly detector evaluates each series against its own history rather than a single fleet-wide baseline. 19
5.5 Fleet-mean backtested WAPE of each series' deployed model, evaluated at each of six historical monitoring checkpoints. No fleet-wide drift is flagged across this window. 22
5.6 The live dashboard, showing history, forecast, confidence band, flagged anomalies, model comparison, and (below the fold) the feature-driver and what-if simulation panels. 23

List of Tables

5.1 Rolling-origin backtest results by model, across 240 series and six cutoffs. 16
5.2 Anomaly label distribution across 48,960 series-weeks. 19
5.3 Empirical 80% prediction-interval coverage, held-out cutoff 2023-10-29 ($n = 1{,}920$ observations). 20
5.4 Top global feature importances (permutation importance, mean absolute error increase). 21

Chapter 1

Introduction

1.1 Background and Motivation

Retail demand forecasting sits at the center of a much larger operating problem than a single accuracy number can capture. A production forecasting system for a multi-store retailer must simultaneously: predict demand far enough ahead to drive inventory and staffing decisions; distinguish an unusual sales week that is fully explained by a promotion or holiday from one that genuinely needs a human's attention; communicate not just a number but a defensible degree of confidence in that number; explain, on request, why the model produced the number it did; and expose all of this to non-technical stakeholders through a natural-language interface without sacrificing auditability. Most forecasting tutorials and portfolio projects address exactly one of these concerns – typically the first – in isolation. This report describes a system built to address all five together, as an integrated pipeline rather than five disconnected notebooks.

1.2 Problem Statement

Given historical weekly sales for a set of (store, department) series, together with store metadata and macroeconomic covariates, the system must: (1) produce an 8-week-ahead forecast for every series, selecting automatically among competing forecasting approaches on a per-series basis; (2) flag statistically unusual weeks in a way that separates explained causes (promotions, calendar holidays) from unexplained ones that warrant investigation; (3) attach a calibration-checked measure of forecast uncertainty to every prediction, not just a point estimate; (4) explain, per forecast, which input features drove the model's output; and (5) expose all of the above through a conversational interface that supports hypothetical ("what if") queries and produces a persisted, inspectable reasoning trace for every answer.

1.3 Research Objectives

- Implement and rigorously backtest multiple forecasting approaches of increasing sophistication (seasonal-naive, classical exponential smoothing, and a global gradient-boosted model), selecting a winner per series rather than assuming one model dominates everywhere.
- Design an anomaly detector that models each series against its own trailing history and explicitly cross-references known causes (active markdowns, calendar holidays) before labeling a deviation "unexplained."
- Attach 80% prediction intervals to every forecast and measure their real coverage against held-out data, rather than asserting calibration.
- Provide model explainability – both local (per-forecast feature attribution) and global (feature importance ranking) – without depending on a library that could not be installed in the target environment.
- Build an agentic natural-language interface with a fully swappable reasoning backend (deterministic mock, Anthropic, or OpenAI) sharing one tool registry, and persist a complete audit trail for every question asked.
- Validate the entire system with an automated test suite and disclose, rather than hide, every substitution and limitation encountered along the way.

1.4 Scope and Limitations

The dataset used throughout this report is synthetic. The original intent was to build directly against the well-known Walmart Recruiting Store Sales Forecasting dataset [1]; this was not possible because the build environment's outbound network access did not permit bulk dataset downloads. Rather than ship a partial or broken download, a schema-identical synthetic generator was built instead, matching the real dataset's column layout, store-size distribution, holiday calendar, and promotional-markdown structure, and additionally injecting a set of known "ground truth" events (a COVID-era demand shock, a localized supply disruption, and data-quality errors) that the rest of the system is validated against directly. This substitution, and every other technical substitution made because of build-environment constraints (gradient boosting library, web framework, exponential-smoothing fitting method, SHAP), is disclosed at the point where it matters rather than collected in a single footnote, and is discussed fully in Chapter 6. The system has not been

deployed against live production traffic or a cloud environment; that gap is stated plainly rather than implied away.

1.5 Report Structure

Chapter 2 surveys the methods this system draws on – forecasting model selection, anomaly detection, prediction intervals, explainability, and agentic tool-calling systems – and identifies the gap this report's integration addresses. Chapter 3 describes the system's architecture and design decisions. Chapter 4 describes the implementation in detail. Chapter 5 reports evaluation results, all drawn directly from the system's own generated artifacts. Chapter 6 discusses the findings, their implications, and the system's limitations. Chapter 7 concludes and outlines future work.

Chapter 2

Related Work

2.1 Retail Demand Forecasting

Multi-step retail demand forecasting has been studied extensively in both the academic and competition literature. The M5 forecasting competition formalized rolling-origin evaluation of hierarchical retail demand at scale and demonstrated that gradient-boosted tree ensembles were, at the time, the most consistently competitive class of model across tens of thousands of heterogeneous series [3]. Classical exponential smoothing methods, including the Holt-Winters seasonal method [6], remain a strong and interpretable baseline, particularly on series with strong, stable annual seasonality where a more flexible model's additional variance is not worth its cost. Hyndman and Athanasopoulos provide the standard reference treatment of both families of method and of backtesting methodology more broadly [2].

2.2 Gradient-Boosted Trees for Tabular Forecasting

Histogram-based gradient boosting frameworks such as LightGBM [4] and XGBoost [5] popularized fast, regularized tree ensembles for large tabular forecasting problems, typically framed as supervised regression over lag, rolling-window, and calendar features rather than as a classical time-series model. Their strength on heterogeneous, high-cardinality retail data is well established; their weakness – a heavier reliance on engineered lag/rolling features and a recursive multi-step forecasting procedure that compounds error along the forecast horizon – is less often discussed in applied work, and is addressed directly in this report's evaluation (Chapter 5).

2.3 Anomaly Detection in Retail Time Series

Time-series anomaly detection commonly combines a decomposition step (trend, seasonal, residual) with a statistical threshold on the residual. Isolation Forest [13] offers a complementary, non-parametric approach that isolates outliers via random recursive partitioning rather than a fitted residual distribution, and is widely used as a cross-check against a primary statistical method rather

than as a sole detector. A recurring, under-discussed failure mode of purely statistical decomposition methods is that calendar-driven seasonality (holidays in particular) does not repeat on an exact fixed-length cycle, so a naive modular seasonal index will systematically misclassify real calendar effects as anomalies – a failure mode this report's system encountered directly and documents the fix for (Section 5.3).

2.4 Prediction Intervals and Uncertainty Quantification

Point forecasts alone understate what a forecasting system actually knows. Quantile regression [7] generalizes ordinary regression to estimate a specified conditional quantile directly, and gradient-boosted trees adapt naturally to a pinball (quantile) loss objective. Conformal prediction [8] offers an alternative, distribution-free route to calibrated intervals with a coverage guarantee under exchangeability, at the cost of additional held-out data. Gneiting and Raftery's treatment of proper scoring rules underscores a point this report treats as central: a prediction interval is only useful if its real coverage is checked against held-out outcomes, not merely asserted by construction [9].

2.5 Model Explainability

SHAP (SHapley Additive exPlanations) [10] is the dominant modern approach to local feature attribution, formalizing attribution as an approximation to Shapley values from cooperative game theory. Permutation importance, following the treatment popularized by Breiman for random forests [11] and formalized model-agnostically by Fisher, Rudin, and Dominici [12], instead measures global importance by the increase in prediction error when a feature is randomly shuffled. Where a SHAP-class library is unavailable, occlusion-based attribution (replacing a single feature with a baseline value and measuring the resulting change in prediction) is a simpler, longer-established approximation to the same idea, with a well-known limitation: it does not average over feature orderings or account for feature interactions the way a true Shapley-value computation does, and can misattribute credit among correlated features.

2.6 Agentic AI and Tool-Calling Systems

Recent large language model systems increasingly separate reasoning from action by giving a model a fixed set of callable tools and an explicit loop in which the model's tool calls are executed and their results fed back before a final answer is produced. ReAct formalized interleaving explicit reasoning traces with tool actions [14], and Toolformer showed that tool-calling behavior can be

learned rather than hand-specified [15]. Both major commercial model providers now expose a structurally similar tool-use interface – the Anthropic Messages API [16] and the OpenAI function-calling / Chat Completions API [17] – which, despite differing request and response shapes, expose the same underlying JSON Schema tool specification, making a provider-agnostic tool registry a natural design choice rather than a compromise.

2.7 Gaps Addressed by This System

Individually, each of the above areas is well studied. What is less commonly built end-to-end in a single system – and what this report's system contributes – is their integration: per-series model selection validated by honest rolling-origin backtesting; anomaly detection that explicitly reconciles its statistical flags against the same calendar and promotional covariates the forecasting model uses; prediction intervals whose calibration is checked against held-out data rather than assumed; explainability that is honest about being a SHAP substitute rather than presented as SHAP; and an agentic interface whose tool-calling backend is swappable across providers without touching the underlying tool implementations or the audit trail format.

Chapter 3

System Design and Architecture

3.1 Architectural Overview

The system is organized around a single load-bearing design decision: a strict separation between an *offline batch* stage, which trains models and scores every forecast and anomaly flag once and writes the results to disk, and a *thin online serving* layer, which only ever reads those artifacts. Figure 3.1 shows the full data flow. This mirrors how forecasting and agentic systems are actually operated at scale: online request latency cannot depend on how expensive the underlying modeling is, and every answer the agent gives is backed by a specific, versioned batch of scored predictions that can be audited after the fact, rather than a live recomputation that might silently differ between two questions asked minutes apart.

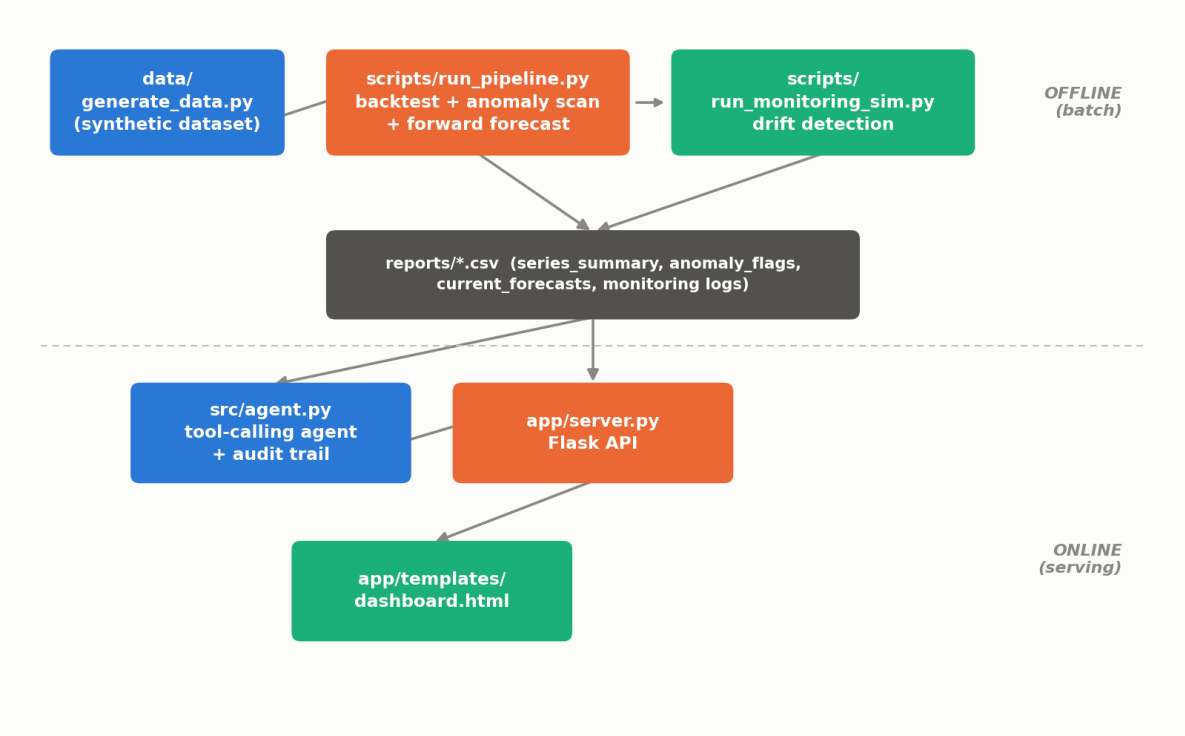

Figure 3.1: End-to-end system architecture. The dashed line separates the offline batch stage (data generation, backtesting, anomaly scanning, forward forecasting, and monitoring simulation) from the thin online serving layer (the agent and the Flask API/dashboard), which only ever reads artifacts the batch stage already wrote.

3.2 Data Pipeline and Feature Engineering

Raw weekly sales, store metadata, and macroeconomic covariates are merged into a single table, sorted by store, department, and date, and expanded with three families of engineered feature: calendar features (week of year, month, sinusoidal week-of-year encoding, and a real US holiday flag), markdown features (total and count of active promotional markdowns), and lag/rolling features (lags at 1, 2, 4, 8, and 52 weeks, and trailing rolling mean/standard deviation over 4-, 8-, and 12-week windows). Every rolling and lag feature is computed strictly from *prior* weeks relative to the row being predicted – a design constraint verified directly by a dedicated leakage-test suite (Section 4.4) – since a single row of look-ahead contamination in a rolling feature can silently inflate backtest accuracy in a way that will not reproduce in production.

3.3 Forecasting Models

Three qualitatively different forecasting approaches are implemented and competed against one another rather than a single model being assumed to dominate.

Seasonal-Naive Baseline

The simplest possible seasonally-aware forecast: the value observed exactly one year (52 weeks) prior. This is included deliberately as a floor that any more sophisticated model must beat to justify its additional complexity.

Holt-Winters Exponential Smoothing

A from-scratch multiplicative triple exponential smoothing implementation [6], fit per series. Smoothing constants are selected via a small fixed grid search minimizing in-sample one-step-ahead squared error, rather than by maximum likelihood, because the standard maximum-likelihood implementation could not be installed in the build environment (Section 6.4 quantifies the resulting accuracy cost).

Global Gradient-Boosted Model

A single histogram-based gradient-boosted regression tree model, trained once across *all* 240 series with store and department included as features rather than as 240 separate per-series models. This mirrors standard practice in large-scale retail forecasting: a global model shares statistical strength across series, generalizes better to sparse or low-volume series that would badly overfit a model trained on their own $\sim$250 data points, and is a single artifact to version, monitor, and retrain. Multi-step forecasts are produced recursively: the model's own prediction at step h is fed back in as a lag feature for predicting step $h+1$, since future actuals are of course unknown at forecast time.

Per-Series Model Selection

Rather than deploying one model everywhere, the winning model for each series is selected independently based on mean weighted absolute percentage error (WAPE) across six rolling-origin backtest cutoffs (Section 4.4) – the "best-fit selection" pattern that avoids a single sophisticated model quietly underperforming a trivial baseline on the subset of series where that baseline is genuinely hard to beat.

3.4 Anomaly Detection Design

Each series is decomposed into a trailing (non-look-ahead) trend estimate and a week-of-year seasonal index, and the residual is scored with a robust, median/median-absolute-deviation-based z-score computed over a fully historical window. A residual flagged as statistically unusual is then

cross-referenced against two known explanatory covariates – an active promotional markdown, and the real US holiday calendar flag – before being labeled. This produces five mutually exclusive categories per series-week: `normal`, `explained_promo_spike`, `explained_holiday_spike`, `data_quality_error` (for implausible values such as negative sales), and `unexplained_anomaly`. A secondary, model-agnostic IsolationForest [13] pass cross-checks the primary statistical flags and marks agreement as "high confidence," giving investigators a coarse triage signal in addition to the underlying label.

3.5 Prediction Interval Design

Every forecast is accompanied by an 80% prediction interval (a 10th-to-90th percentile band), constructed differently depending on which model won the backtest for that series. For global-GBM series, two additional gradient-boosted models are trained with pinball (quantile) loss at $q = 0.10$ and $q = 0.90$ [7] and queried along the point model's own recursive forecast path. For seasonal-naive and Holt-Winters series, which have no natural quantile-regression analogue, the interval is instead a normal-approximation band scaled by that series' own backtested root-mean-squared error. Both methods are, by design, approximations rather than provably calibrated procedures, so their real coverage is measured directly against held-out data (Section 5.4) rather than assumed.

3.6 Explainability Design

Local, per-forecast feature attribution is implemented via occlusion: each feature in a specific forecast's input row is replaced, one at a time, with its global median value (computed once across the full training set), and the resulting change in the model's prediction is recorded as that feature's impact on that specific forecast. Global feature importance is implemented via permutation importance [11, 12]: each feature column is independently shuffled and the resulting increase in mean absolute error is measured, averaged over several repeats. Both methods are used in place of SHAP [10], which could not be installed in the build environment, and both are explicitly documented as approximations rather than presented as equivalent to a true Shapley-value computation.

3.7 Agent Architecture and Tool Design

The natural-language interface is implemented as a small tool-calling agent over six discrete, typed tools: `get_forecast`, `get_anomalies`, `explain_change`, `top_movers`, `explain_forecast_drivers`, and `simulate_scenario`. Five of the six tools only ever read artifacts the offline batch stage al-

ready produced; the sixth, `simulate_scenario`, is a deliberate exception, since a hypothetical covariate combination ("what if this series ran a markdown promotion these next three weeks?") was by definition never scored offline, and is instead recomputed live against the persisted point model. The reasoning "brain" behind tool selection is itself swappable: a deterministic mock router (regex/keyword based, requiring no API key, used for tests and CI), a real tool-calling loop against the Anthropic Messages API [16], and a real tool-calling loop against the OpenAI function-calling API [17], all sharing one tool registry and one JSON Schema tool specification (converted, not duplicated, between the two providers' differing request shapes). Every tool call, regardless of which backend selected it, is logged to a persisted `AgentTrace` – the question asked, each tool invoked with its arguments and result, and the final answer – giving every answer a complete, inspectable audit trail.

3.8 Dashboard Frontend

A browser dashboard, served by a thin Flask API layer, renders the same artifacts the agent reads: an interactive history-plus-forecast chart with a shaded 80% confidence band, a per-series flagged-anomaly table, a model-performance comparison chart, a feature-driver panel (for global-GBM series), a what-if scenario control panel, and a chat panel that exposes the agent's full reasoning trace for every answer. All charts are rendered from scratch in inline SVG with no external JavaScript dependency, so the dashboard has no build step and no content delivery network dependency.

Chapter 4

Implementation

4.1 Development Environment

The system is implemented in Python 3.11, using pandas and NumPy for data handling, scikit-learn's histogram-based gradient boosting implementation for the global model and its quantile variants, Flask for the API layer, and plain HTTP (via the `requests` library) rather than a vendor SDK for both LLM tool-calling backends. The automated test suite uses the Python standard library's `unittest` module. Several implementation choices were constrained by the build sandbox's restricted outbound network access, and are documented at the point where they matter rather than collected separately: LightGBM was substituted with scikit-learn's `HistGradientBoostingRegressor` (same algorithm family: histogram-based gradient-boosted trees); FastAPI was substituted with Flask (identical REST semantics, no async support); and statsmodels' maximum-likelihood exponential smoothing was substituted with a fixed-grid-search implementation (Section 3.3.2).

4.2 Synthetic Data Generation

Because the build environment could not reach the real Walmart Recruiting dataset [1], a schema-identical synthetic generator produces 20 stores, 12 departments, and 260 weeks of history (62,400 rows), matching the real dataset's store-size distribution and the same four US holidays it flags (Super Bowl, Labor Day, Thanksgiving, Christmas). Beyond matching schema and statistical character, the generator deliberately injects a set of known ground-truth events used later purely for validation: a COVID-era demand shock (March–August 2020) that is *asymmetric by department* (some departments spike, others crash, mirroring real 2020 retail behavior); a single, localized five-week supply disruption for one store/department pair with no promotional, holiday, or macro cause; and a small number of randomly scattered data-entry errors (negative sales and implausible spikes). Every one of these injected events is later checked against what the anomaly detector actually flags (Section 5.3).

4.3 Feature Engineering Implementation

Lag and rolling features are computed with a shift-then-rolling pattern (shifting the target series by one row before computing any rolling statistic) rather than a naive rolling-then-shift pattern, specifically to guarantee that no rolling window can see the value it is being used to predict. This constraint is enforced by three dedicated unit tests verifying, respectively, that a lag-1 feature equals the truly previous row (not the current one), that a rolling mean excludes the current row, and that per-series feature computation does not bleed across a store/department boundary when the underlying table is sorted globally by date.

4.4 Rolling-Origin Backtest Implementation

Six rolling-origin cutoffs, spaced eight weeks apart across the most recent roughly 48 weeks of history, are used to backtest all three forecasting models against all 240 series simultaneously. At each cutoff, the global gradient-boosted model is retrained fresh on everything strictly before that cutoff (never on data from or after it), and all three models forecast the following eight weeks; each model's forecast is then scored against the true, held-out values with WAPE, mean absolute percentage error, and root-mean-squared error. The winning model per series is the one with the lowest mean WAPE across all six cutoffs (Section 5.2).

4.5 Prediction Interval Implementation

Two quantile-loss gradient-boosted models ($q = 0.10$ and $q = 0.90$) are trained once on the full available history and persisted to disk alongside the point model, so that the dashboard and agent reuse the *exact* trained model that produced the baseline forecast rather than silently retraining a slightly different one. Because two independently trained quantile models carry no mathematical guarantee that the $q = 0.10$ prediction stays below the $q = 0.90$ prediction at every point ("quantile crossing"), the implementation explicitly enforces this ordering, and additionally guarantees the point forecast always falls inside its own interval, rather than silently emitting a mathematically invalid band. Calibration is checked by refitting all three GBM variants on data strictly before the single most recent backtest cutoff and measuring what fraction of the real, never-trained-on future values fall inside the resulting interval (Section 5.4).

4.6 Explainability Implementation

The trained point model, both quantile models, and the full set of per-feature training medians are persisted via `joblib` immediately after training, so that local occlusion attribution and global permutation importance are always computed against the identical model instance that produced the corresponding forecast. Global permutation importance is computed on a fixed 6,000-row random sample of the training table (rather than the full ∼60,000-row table), an explicit, disclosed speed/precision tradeoff rather than a silent truncation.

4.7 What-If Scenario Simulation

The what-if tool recomputes the global gradient-boosted model's recursive forecast under a hypothetical override of one or both of two covariates: whether a promotional markdown is active, and whether the forecasted weeks should be treated as a holiday. A specific implementation detail matters here and was corrected during development: the holiday override defaults to *not touching the real calendar* rather than defaulting to "no holiday." An earlier version defaulted the holiday flag to false whenever a caller only asked about a markdown, which silently removed a genuine holiday effect from any week that happened to actually be a real holiday, producing a misleading result that looked like "the markdown caused a large drop" when the actual cause was an inadvertently deleted holiday spike. This is now covered by a dedicated regression test (Section 4.9).

4.8 Agent Assembly and Tool Wiring

All six tools are registered in a single dictionary keyed by tool name, alongside one shared JSON Schema tool specification list. The Anthropic and OpenAI backends both consume this same specification list – the OpenAI variant via a small, purely structural conversion function, since both providers' underlying schemas are JSON Schema, differing only in how the surrounding request object wraps them. Backend selection follows an explicit precedence: an `LLM_BACKEND` environment variable, when set, always wins; otherwise, whichever provider API key is present is used (Anthropic preferred if both are present); and if neither is present, the deterministic mock backend is used, guaranteeing the system always runs end-to-end without requiring any external credential.

4.9 API and Frontend Implementation

A Flask application exposes the forecast, anomaly, evaluation-summary, driver-explanation, and scenario-simulation data as JSON endpoints, plus a single natural-language question-answering endpoint that invokes the agent and returns its complete trace. The dashboard is a single self-contained HTML document with inline SVG charting and no external JavaScript dependency, verified with a headless-browser check that confirms zero console errors during a full interaction sequence (selecting a series, loading its forecast and confidence band, viewing its driver panel, and running a what-if scenario).

4.10 Testing and Validation

The full system is validated by 49 automated unit tests, covering: the three forecasting models in isolation; the anomaly detector against synthetic injected events; the three explicit no-look-ahead-leakage guarantees in feature engineering; the agent's tool functions and its deterministic routing logic; the backend-selection precedence logic (mocked, without any network call); the prediction-interval assembly logic, including the quantile-crossing correction; the occlusion and permutation-importance explainability functions against synthetic data with a known ground-truth answer; and the what-if scenario simulator's calendar-handling correctness, including a regression test for the holiday-defaulting bug described in Section 4.6.

Chapter 5

Evaluation and Results

5.1 Evaluation Framework

All figures and numbers reported in this chapter are produced directly by the system's own pipeline scripts and are reproduced automatically every time those scripts are run; none are hand-adjusted or rounded favorably.

5.2 Forecasting Accuracy

Table 5.1 and Figures 5.1–5.2 summarize the rolling-origin backtest results across all 240 series and six cutoffs.

Table 5.1: Rolling-origin backtest results by model, across 240 series and six cutoffs.

Model	Mean WAPE	Mean MAPE	Series won (of 240)
Global gradient-boosted model	**8.21%**	9.30%	**140**
Seasonal-naive	8.58%	9.50%	90
Holt-Winters (fixed-parameter)	11.16%	12.26%	10

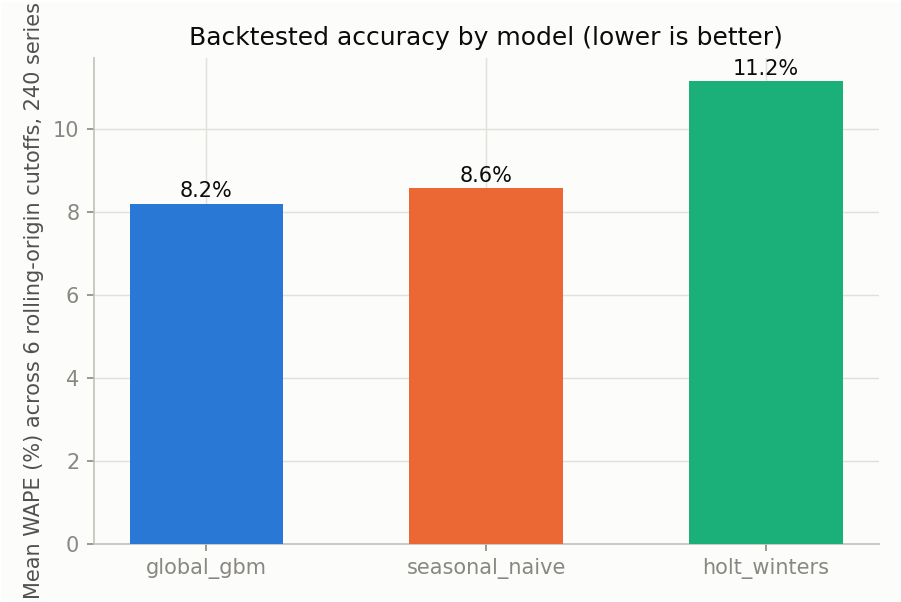

Figure 5.1: Mean backtested WAPE by model across all 240 series.

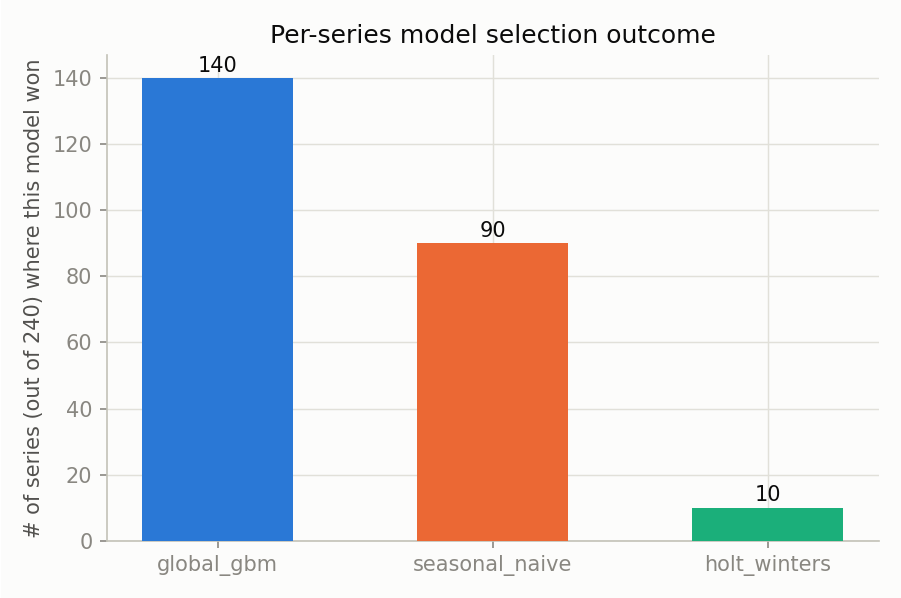

Figure 5.2: Number of series (of 240) on which each model achieved the lowest backtested WAPE.

The global gradient-boosted model wins both on mean WAPE and on raw series count, but by less than half a percentage point of WAPE over the seasonal-naive baseline – and the baseline still wins outright on 90 of 240 series (37.5%). This is reported as a genuine, expected result rather than a shortcoming: those 90 series are overwhelmingly ones with strong, low-noise annual seasonality, where "the same week last year" is close to the ceiling of what is predictable, and a more flexible model's extra variance costs more than it is worth. This is precisely why the system

selects a winning model per series rather than deploying the most sophisticated model everywhere. Figure 5.3 shows one representative series (Store 3, Department 5) where the global GBM was selected, illustrating both the forecast and two anomalies flagged in its recent history.

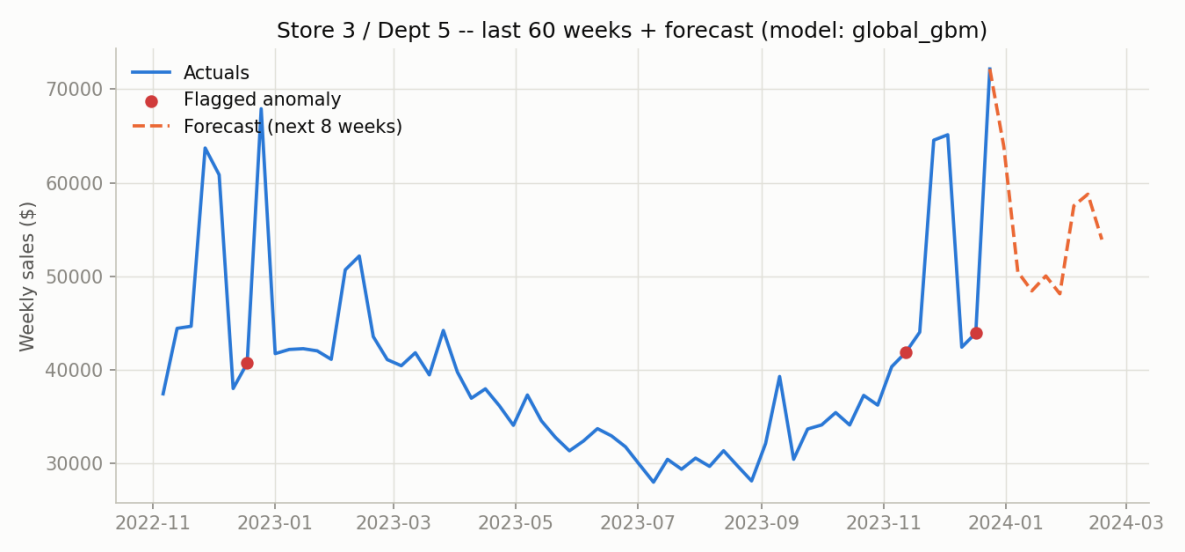

Figure 5.3: Example series (Store 3, Department 5): trailing 60 weeks of actuals, two flagged anomalies, and the 8-week forward forecast under the selected global gradient-boosted model.

Holt-Winters' comparatively weak result traces directly to the disclosed substitution in Section 3.3.2: fixed-grid smoothing constants instead of maximum-likelihood-fit ones. This is a real, measured accuracy cost, not a hidden one.

5.3 Anomaly Detection Validation

Table 5.2 summarizes the anomaly labels assigned across all 48,960 series-weeks, and the detector's validation against the generator's known, injected ground-truth events.

Table 5.2: Anomaly label distribution across 48,960 series-weeks.

Category	Count
Normal	45,028
Unexplained anomaly (needs investigation)	2,653
Explained – calendar holiday spike	895
Explained – promotional markdown spike	323
Data-quality error (negative/implausible value)	61
High-confidence (statistical + IsolationForest agree)	473

Against the generator's injected ground-truth events specifically: 100% (61 of 61) of synthetic data-entry errors are correctly flagged as `data_quality_error`; 4 of 5 weeks of the localized, unexplained supply disruption are flagged as needing investigation; and 211 of 240 series show at least one flagged week during the injected COVID-era shock window, consistent with a shock whose sign and magnitude genuinely vary by department (Figure 5.4) rather than every series crossing a single fixed detection threshold.

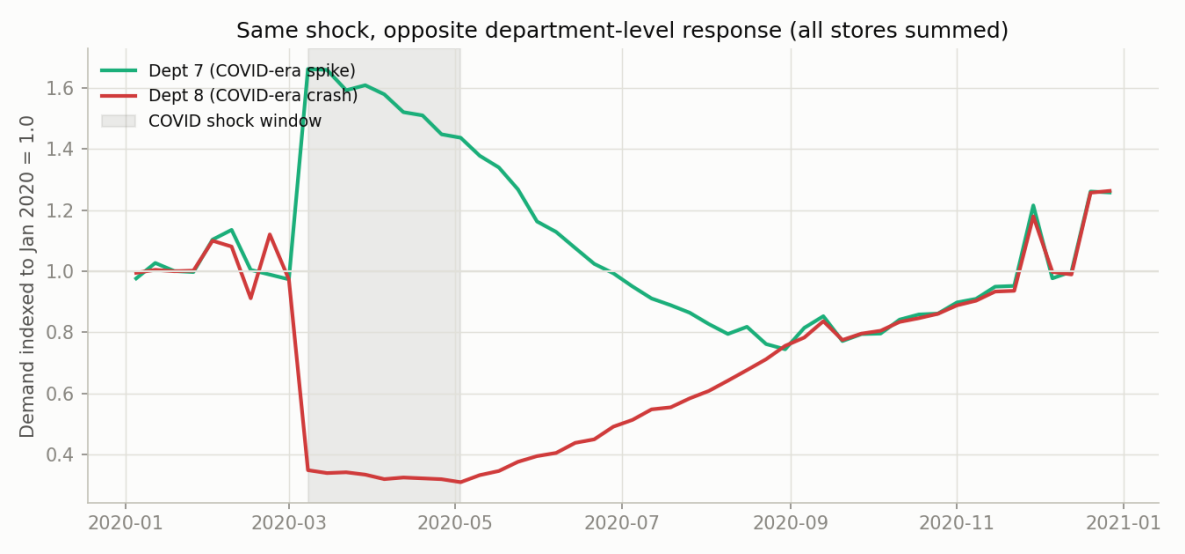

Figure 5.4: The same injected macro shock produces opposite department-level responses – one department spikes while another crashes – which is why the anomaly detector evaluates each series against its own history rather than a single fleet-wide baseline.

5.3.1 A Limitation Found and Fixed

An earlier version of the anomaly detector used a naive modular week-of-year seasonal index, which does not perfectly align with real calendar holidays (Thanksgiving, Labor Day, and similar holidays do not repeat on an exact 52-week cycle). That version misclassified a meaningful number of ordinary December holiday spikes as unexplained anomalies. The fix – cross-checking every statistically flagged point against the same real holiday-calendar flag the forecasting model already uses – reduced false "unexplained" flags from 3,548 to 2,653 without weakening detection of the real injected disruption or the COVID-era shift. This is reported as a concrete illustration of a general principle: a decomposition-based anomaly detector that only examines its own residual will systematically misfire on real calendars, because real calendars do not repeat on clean, fixed cycles.

5.4 Prediction Interval Calibration

Table 5.3 reports the empirical, held-out coverage of the system's 80% prediction intervals, measured by refitting on data strictly before the most recent backtest cutoff and checking what fraction of the true, never-trained-on future values fall inside the resulting band.

Table 5.3: Empirical 80% prediction-interval coverage, held-out cutoff 2023-10-29 (n = 1,920 observations).

Selected model	Empirical coverage
Seasonal-naive	77.5%
Global gradient-boosted model	73.0%
Holt-Winters	72.5%
Overall	**74.7%**

Against a nominal target of 80%, all three methods run somewhat narrow – real coverage falls 5 to 8 percentage points short. This is measured and reported directly rather than assumed away, and has a plausible, disclosed cause for each method: the RMSE-normal-approximation band assumes constant, Gaussian error variance across the full eight-week horizon, when real forecast error typically grows with the number of weeks ahead; and the gradient-boosted quantile band is queried along the point model's own single recursive forecast path rather than branching a full recursive quantile tree, understating how uncertainty compounds recursively. Section 6.4 discusses the calibration fix a next iteration would apply.

5.5 Feature Importance and Explainability Results

Table 5.4 reports global permutation importance for the gradient-boosted model, computed on a 6,000-row sample of the training table with three repeats.

Table 5.4: Top global feature importances (permutation importance, mean absolute error increase).

Feature	Importance (MAE increase)
`rollmean_4` (trailing 4-week rolling mean)	6,560.9
`lag_1` (previous week's sales)	5,458.9
`IsHoliday` (calendar holiday flag)	1,615.2
`lag_52` (same week, prior year)	1,607.2
`rollmean_12` (trailing 12-week rolling mean)	532.8
`rollmean_8` (trailing 8-week rolling mean)	450.7

The model leans heavily on recent short-window history: the two most important features by a wide margin are the trailing 4-week rolling mean and the immediately preceding week's value. `IsHoliday` and `lag_52` (the same week one year prior) matter, but roughly a third as much as recent history. This ranking is computed in-sample, since the final serving model is trained on all available history and there is no leftover held-out slice to score against without shrinking the training set – disclosed as a limitation rather than presented as a held-out result.

5.6 Production Monitoring Simulation

Using each series' own currently *deployed* model's backtested accuracy at each of the six rolling-origin cutoffs as a stand-in for a new week of ground truth arriving, no fleet-wide accuracy drift is detected across the evaluation window (Figure 5.5). However, checking each series against its *own* history, rather than only the fleet average, surfaces four individual series whose deployed model's accuracy degraded sharply enough (an 8-point-or-greater WAPE jump) to warrant a retraining review, even though the fleet-wide average looked stable throughout.

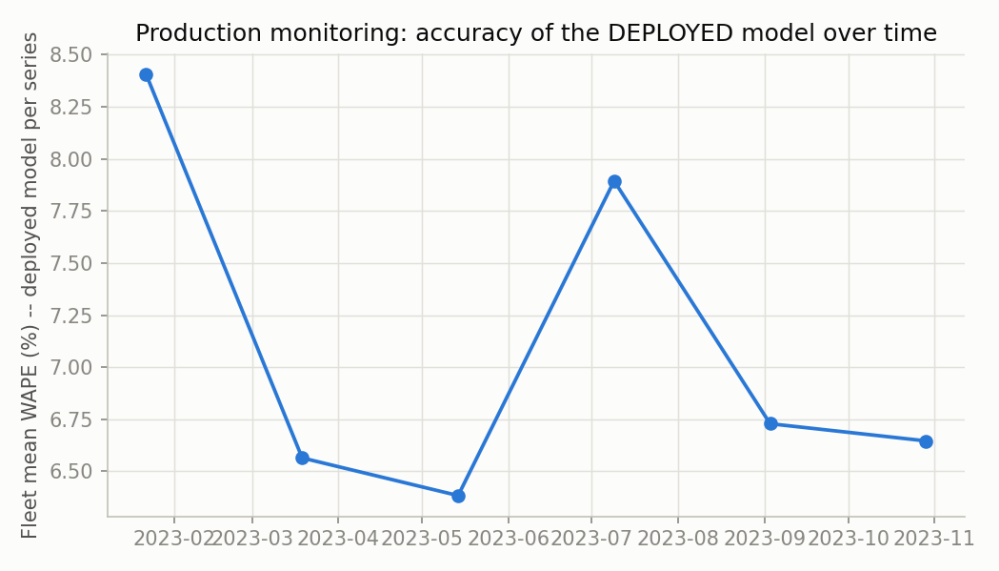

Figure 5.5: Fleet-mean backtested WAPE of each series' deployed model, evaluated at each of six historical monitoring checkpoints. No fleet-wide drift is flagged across this window.

This is the central point of monitoring at the series level rather than only the fleet level: "the average looks fine" and "every individual series is fine" are different claims, and a system that only checks the first is precisely how a handful of series are allowed to silently go stale in production.

5.7 Dashboard and Agent Demonstration

Figure 5.6 shows the live dashboard for a representative series, including the shaded 80% confidence band on the forecast chart, the flagged-anomaly table, the model-performance comparison, the occlusion-based feature-driver panel, and the what-if scenario control.

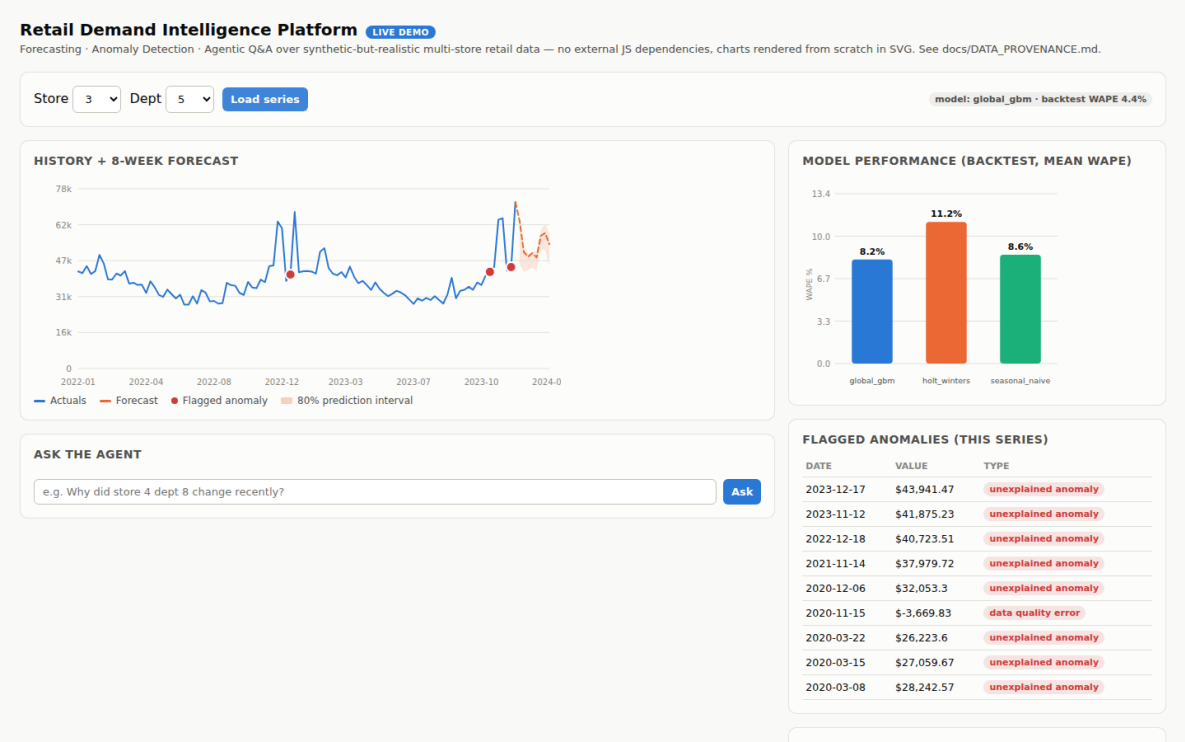

Figure 5.6: The live dashboard, showing history, forecast, confidence band, flagged anomalies, model comparison, and (below the fold) the feature-driver and what-if simulation panels.

Chapter 6

Discussion

6.1 Interpretation of Findings

Three findings stand out. First, the marginal accuracy advantage of the global gradient-boosted model over the seasonal-naive baseline (under half a WAPE percentage point) is a genuinely useful result precisely because it is unflattering: it demonstrates that per-series model selection, rather than blanket deployment of the most sophisticated model, is the correct design, since a third of all series are better served by the simpler baseline. Second, the anomaly detector's holiday-misclassification bug, and its fix, generalizes beyond this system: any decomposition-based detector relying on a fixed-length seasonal cycle will systematically misfire on real calendars. Third, the measured prediction-interval undercoverage (74.7% against an 80% target) is itself a useful evaluation result, not a failure to hide – it identifies specifically which of the two interval-construction methods needs recalibration and by roughly how much.

6.2 Comparison with Related Approaches

Relative to a typical single-model forecasting notebook, this system's distinguishing choices are the per-series backtested model selection (Section 3.3.4), the explicit reconciliation of statistical anomaly flags against known covariates (Section 3.4) rather than a bare statistical threshold, and the pairing of every forecast with both an uncertainty estimate and, where available, a feature-level explanation. Relative to a typical chatbot-style LLM demo, the agent here is deliberately constrained to a fixed, auditable tool registry rather than open-ended generation, and every tool call – regardless of which reasoning backend selected it – produces an identical, persisted trace format.

6.3 Practical Implications

The offline/online architectural split (Chapter 3) is the same shape a production deployment onto a managed batch scheduler and a lightweight serving layer would take, without requiring any change

to the modeling or agent code itself. The explicit backend-selection precedence for the agent (Section 4.8) means the same tool-calling logic and audit-trail format would work unchanged behind either major commercial LLM provider, which matters in any setting where vendor flexibility, cost, or availability considerations might require switching providers without a system rewrite.

6.4 Limitations

Several limitations are disclosed directly rather than implied away. The dataset is synthetic, not real retail data (Section 6.5 discusses this further). Future macro covariates (temperature, fuel price, CPI, unemployment) are held at their last observed value for the forward forecast rather than forecast themselves, a simplifying assumption a production system would replace with its own macro forecasts. The Holt-Winters implementation uses a fixed grid search rather than maximum-likelihood fitting, at a measured accuracy cost (Section 5.2). Prediction intervals run narrower than their nominal target (Section 5.4); a calibration multiplier fit against the same held-out check, or a move to a formal conformal-prediction procedure, would close most of this gap. Global feature importance is computed in-sample rather than against a genuinely held-out slice. The system's continuous integration workflow and containerized deployment were authored to standard, unexotic patterns but could not be exercised against a live GitHub Actions runner or Docker daemon from within the build sandbox, so "written correctly" and "observed passing" are kept as distinct, separately stated claims. Finally, the production WSGI server swap (gunicorn, in place of the Flask development server) is documented with exact commands but was not validated end-to-end, since gunicorn itself could not be installed in the same sandbox.

6.5 Data and Ethical Considerations

Because the dataset is synthetic, no conclusion in this report should be read as a claim about real Walmart, or any other retailer's, actual sales behavior; every document in this project states this plainly, and the synthetic generator's schema and injected events are fully described (Section 4.2) specifically so that swapping in a real, licensed dataset requires no change to any downstream code. More generally, any deployed version of a system like this should treat its anomaly flags as a triage aid for a human investigator, not an automated decision-maker, particularly given the measured false-flag rate discussed in Section 5.3 and the measured prediction-interval undercoverage discussed in Section 5.4.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This report has described the design, implementation, and evaluation of an integrated retail demand intelligence platform combining per-series backtested model selection, explanation-aware anomaly detection, calibration-checked prediction intervals, disclosed explainability substitutes, and a fully auditable, provider-agnostic agentic interface. Every quantitative result reported – the 8.21% mean backtested WAPE, the 90-of-240 series where a trivial baseline still wins, the 74.7% measured (not assumed) prediction-interval coverage, the specific anomaly-detector bug found and fixed mid-development, and the 49 passing automated tests – is produced directly by the system's own pipeline and reproducible by running it again. The recurring design principle across every component is the same: report the real number, disclose the real limitation, and let the evaluation stand on its own rather than on how it is described.

7.2 Future Work

Several concrete extensions follow directly from the limitations disclosed in Section 6.4: fitting Holt-Winters smoothing constants by maximum likelihood rather than a fixed grid; forecasting the macro covariates themselves rather than holding them at their last observed value; calibrating the prediction intervals against the held-out coverage check already implemented, or replacing them with a formal conformal-prediction procedure; recomputing global feature importance against a genuinely held-out data slice; extending the anomaly detector with explicit date-distance-to-holiday features rather than a same-week boolean cross-check; replacing the single fixed anomaly-detection threshold with a per-series-calibrated one; and wiring the production monitoring simulation to a real scheduler against live incoming weekly data rather than replaying historical backtest cutoffs. Beyond the existing scope, a natural next step is validating the containerized deployment and continuous-integration workflow against a live Docker daemon and GitHub Actions runner, closing the one remaining gap between "written to standard patterns" and "observed passing in production infrastructure."

Bibliography

[1] Kaggle. (2014). *Walmart Recruiting – Store Sales Forecasting* [Data set]. https://www.kaggle.com/c/walmart-recruiting-store-sales-forecasting

[2] Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts.

[3] Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022). M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, 38(4), 1346–1364.

[4] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.

[5] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

[6] Winters, P. R. (1960). Forecasting sales by exponentially weighted moving averages. *Management Science*, 6(3), 324–342.

[7] Koenker, R., & Bassett, G. (1978). Regression quantiles. *Econometrica*, 46(1), 33–50.

[8] Vovk, V., Gammerman, A., & Shafer, G. (2005). *Algorithmic Learning in a Random World*. Springer.

[9] Gneiting, T., & Raftery, A. E. (2007). Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477), 359–378.

[10] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.

[11] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

[12] Fisher, A., Rudin, C., & Dominici, F. (2019). All models are wrong, but many are useful: Learning a variable's importance by studying an entire class of prediction models simultaneously. *Journal of Machine Learning Research*, 20(177), 1–81.

[13] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. *Proceedings of the 8th IEEE International Conference on Data Mining*, 413–422.

[14] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*.

[15] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36.

[16] Anthropic. (2024). *Messages API and tool use* [Technical documentation]. https://docs.anthropic.com

[17] OpenAI. (2024). *Function calling and tool use guide* [Technical documentation]. https://platform.openai.com/docs

[18] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.

[19] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Appendix A

Repository Structure

data/	synthetic data generator, generated CSVs, ground-truth event log
src/	forecasting, backtest, anomaly, features, intervals, explain, scenario, and the tool-calling agent
scripts/	orchestration entry points: pipeline, monitoring simulation, report figures, demo assets
app/	Flask API and the browser dashboard (no external JS dependency)
tests/	49-test automated suite (Python standard library unittest)
reports/	generated artifacts: metrics, figures, agent trace log, interval-coverage and feature-importance reports, persisted model files
docs/	architecture, evaluation, data-provenance, and job-mapping documentation

Appendix B

Agent Tool Registry

```
get_forecast(store, dept)
    -> next-8-week forecast, selected model, backtested WAPE

get_anomalies(store, dept, limit)
    -> flagged anomalies for one series

explain_change(store, dept)
    -> reasoned explanation of the most recent value (promo / anomaly /
       normal trend-seasonal), with year-over-year context

top_movers(direction, n)
    -> series ranked by year-over-year percent change

explain_forecast_drivers(store, dept, week_ahead)
    -> occlusion-based local feature attribution for one forecasted week
       (global-GBM-selected series only)

simulate_scenario(store, dept, markdown_active, is_holiday, weeks)
    -> live recomputed forecast under a hypothetical covariate override,
       compared against the baseline (global-GBM-selected series only)
```